Lecture 04:

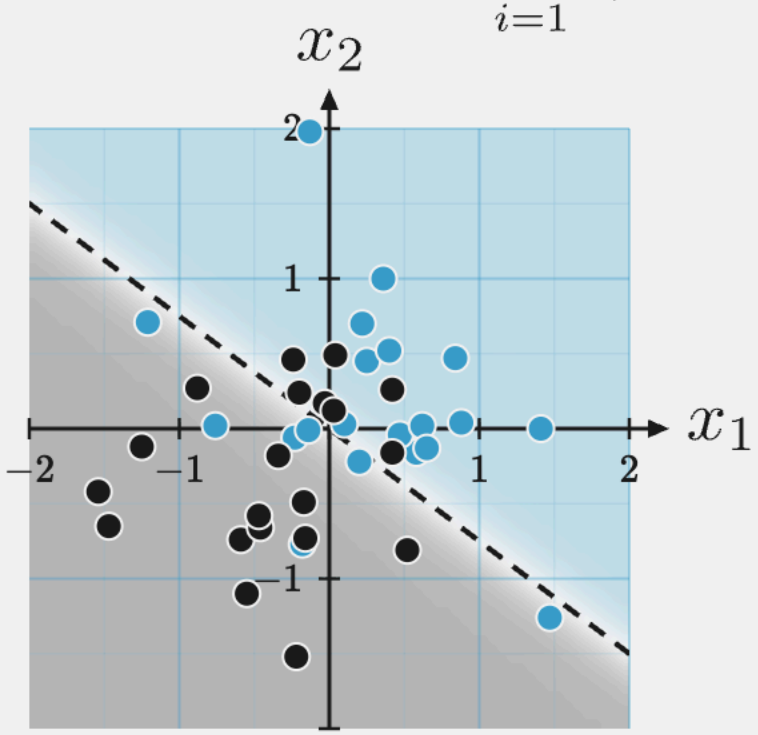
Deep Learning and Automatic Differentiation

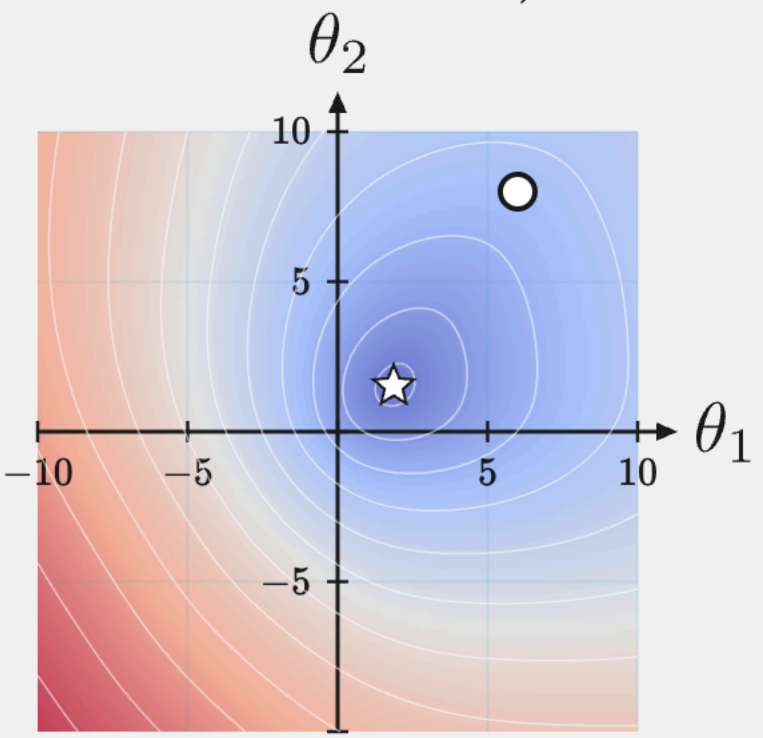
Matthew Wicker

15 October 2026

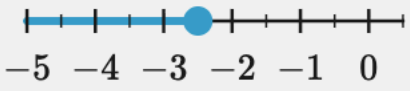
Recalling logistic regression

$$\mathcal{L}(\theta) = -\sum_{i=1}^{m} \Big(y_i \log \sigma(\theta^\top \boldsymbol{x}_i) + (1 - y_i) \log \big(1 - \sigma(\theta^\top \boldsymbol{x}_i)\big) \Big)$$





$log_{10}\,\alpha = -2.50$



$log_{10}\,k = 3.0$



$t = 0,\ \theta = (6, 8),\ \mathcal{L} = 36.6$

$\alpha = 0.0032,\ k = 1000$

Generalized linear model

$$\boldsymbol{y} = g^{-1}\left(\theta^{\top}\phi(\boldsymbol{x})\right)$$

Output

Link Function

Parameter Vector

Input + Basis Expansion

Gradient Descent

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)})^\top$$

Updated parameter, after t+1 “steps”

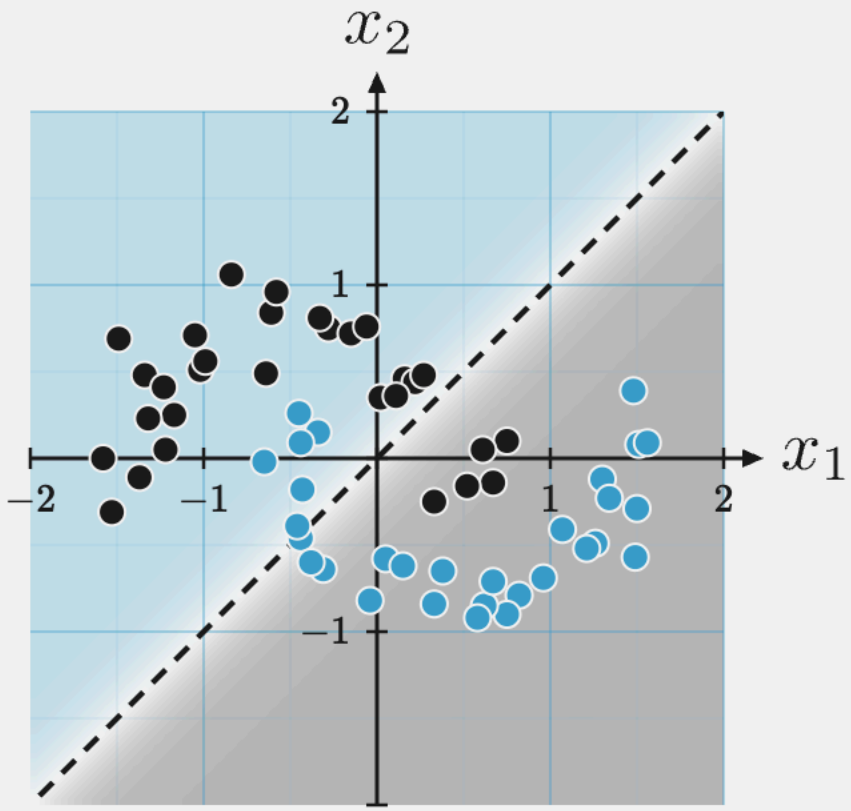
Learning rate

Grad. wrt. the params

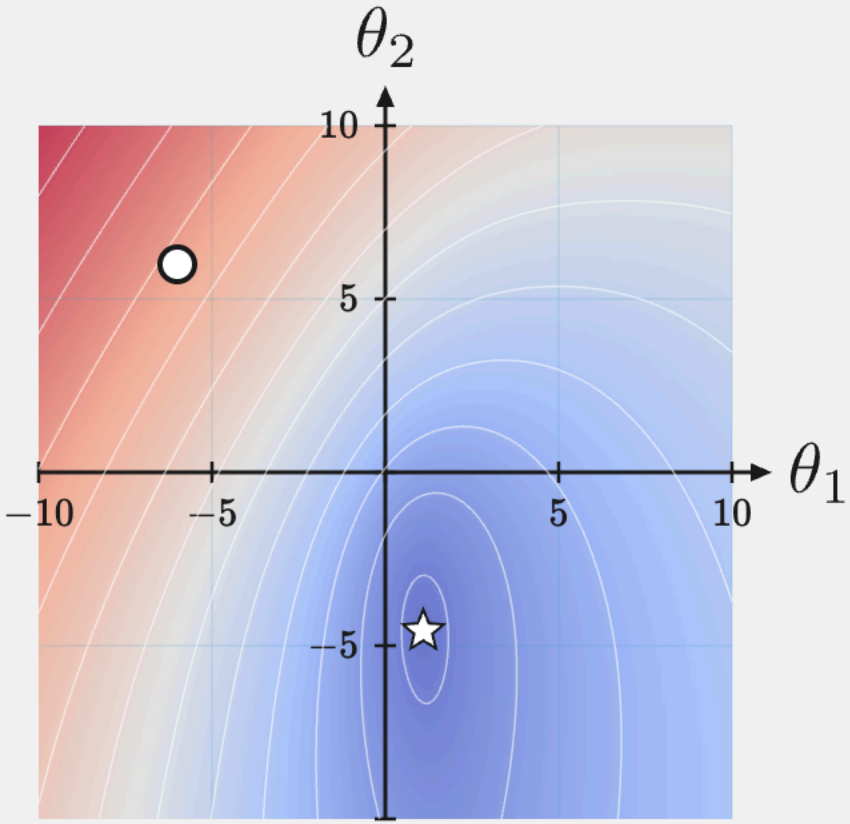
Loss landscape

Logistic regression on the half moons

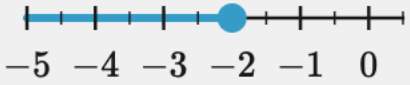
function space: drag or click the data



parameter space



$log_{10}\,\alpha = -2.00$



$log_{10}\,k = 3.0$



$t = 0,\ \theta = (-6, 6),\ \mathscr{L} = 362.8,$ accuracy 18%

$\alpha = 0.01,\ k = 1000$

Towards a graphical representation

Replace our “link” function with what we will now call an “activation function”

$$\text{Let } g^{-1} = h$$

Define our basis expansion to be the affine basis expansion (where we just add a bias):

$$\text{Let } \phi(\boldsymbol{x}) = [\boldsymbol{x}^\top, 1]^\top, \quad \phi : \mathbb{R}^n \to \mathbb{R}^{n+1}$$

Towards a graphical representation

We make the additional assumption here that we only have a single dimensional output we go from:

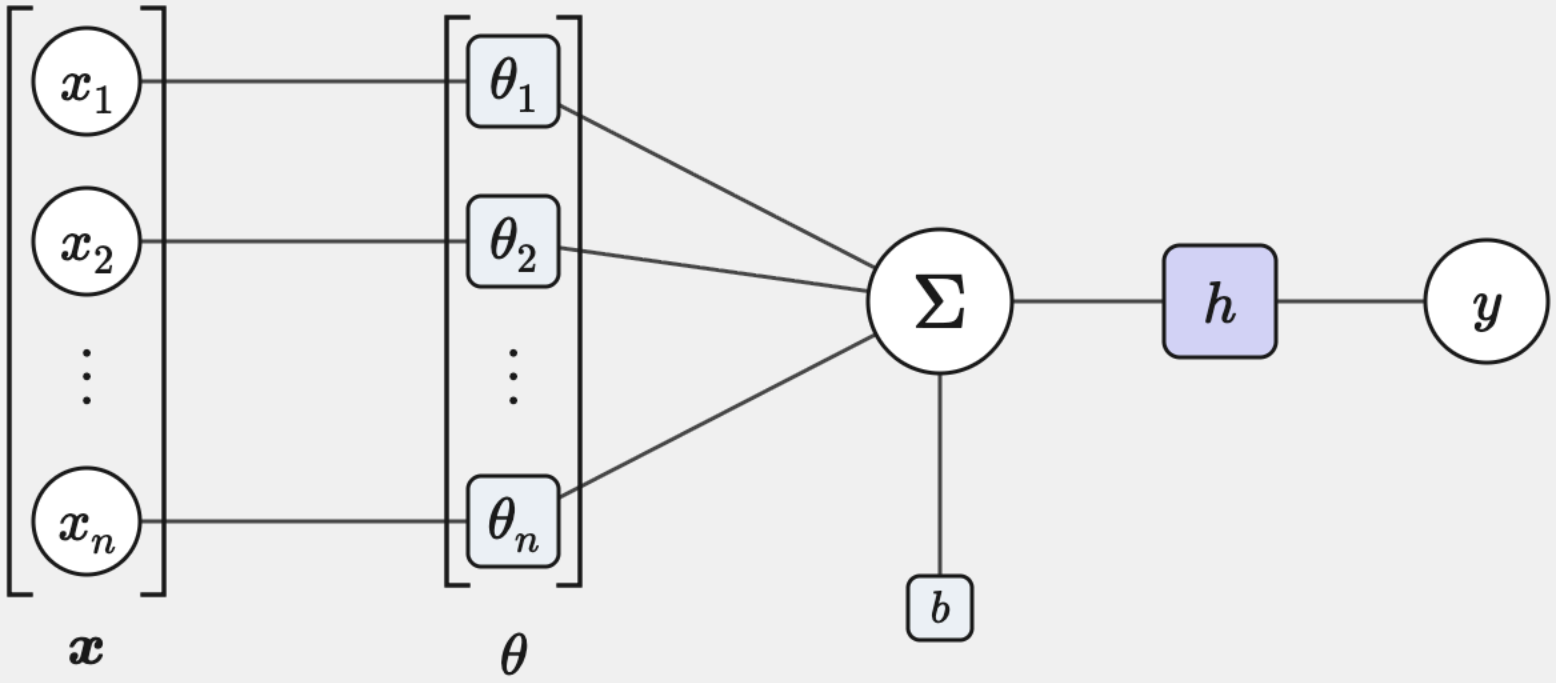
$$\boldsymbol{y} = g^{-1}\left(\theta^{\top}\phi(\boldsymbol{x})\right)$$

expressed as index notation, and for the time being re-writing the link function g^{-1} to just h:

$$y = h\left(b + \sum_{i} \theta_i x_i\right)$$

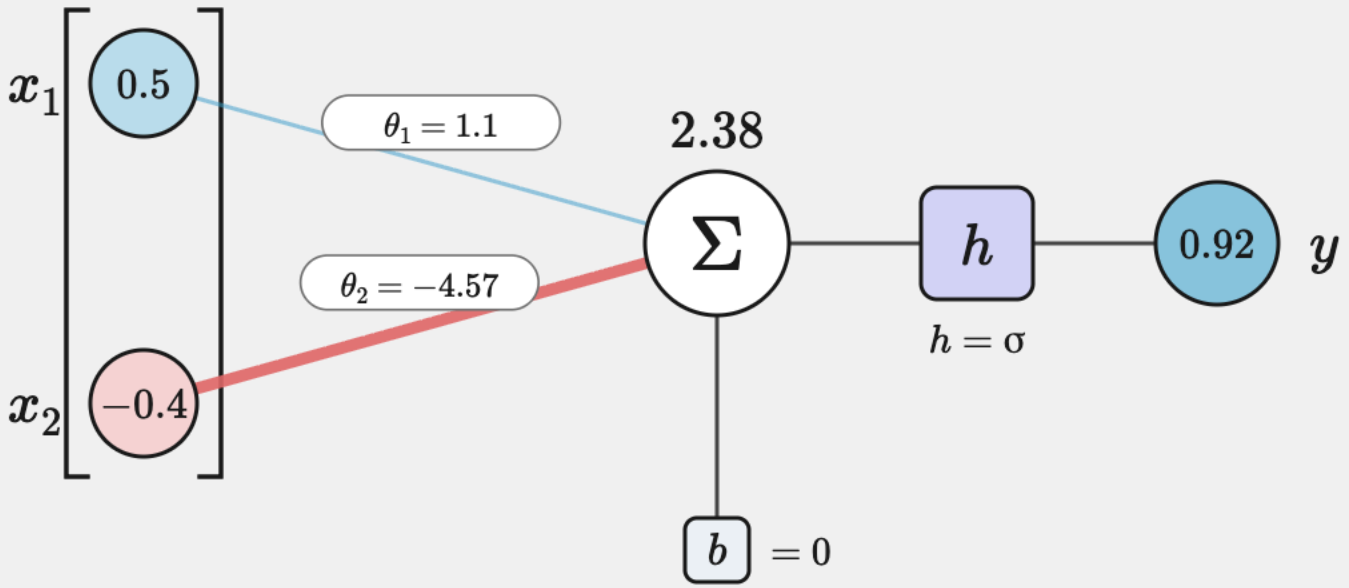
Towards a graphical representation

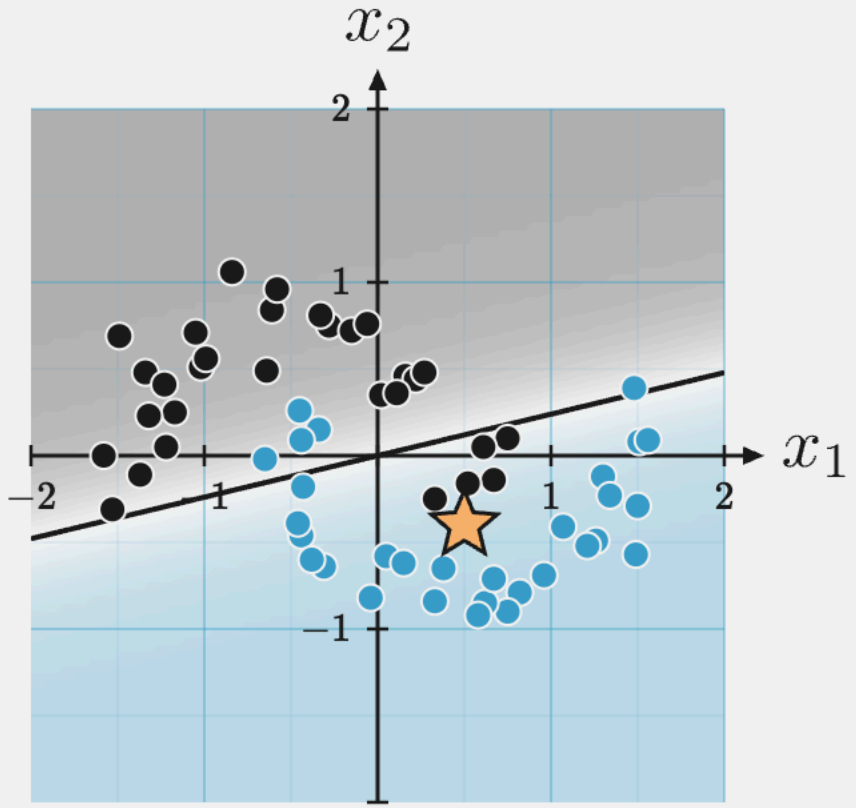
$$y = h\left(b + \sum_i \theta_i \, x_i\right)$$



Towards a graphical representation

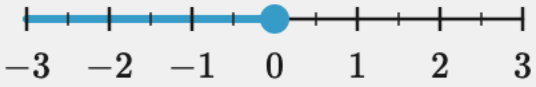
$$y = h\left(b + \sum_i \theta_i \, x_i\right)$$





$$y = \sigma(b + \theta_1 x_1 + \theta_2 x_2) = \sigma(2.38) = 0.92$$

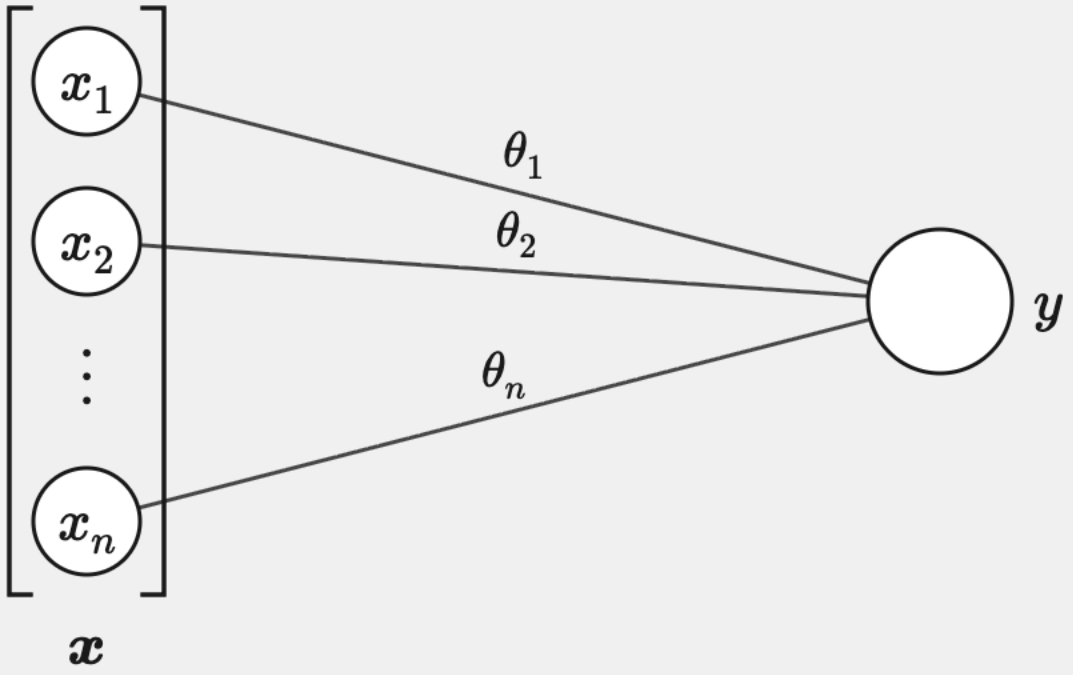
$b = 0.00$

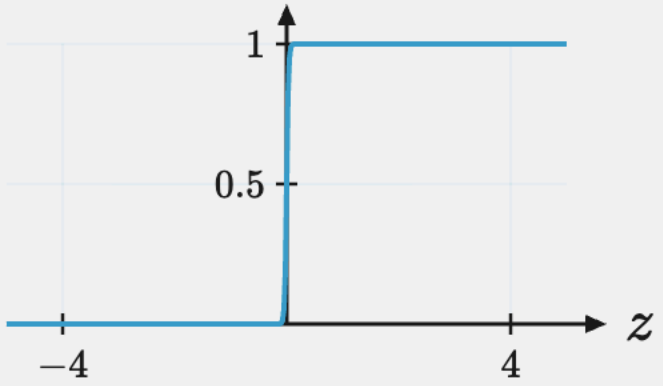


The neuron

What we have built is a **neuron**:

$$y = h\left(b + \sum_i \theta_i \, x_i\right)$$





$$h(z) = \begin{cases} 1 & z > 0 \\ 0 & \text{otherwise} \end{cases}$$

With *h* the step function this is the threshold neuron of McCulloch and Pitts (1943); Rosenblatt's **perceptron** (1957–58) learned its weights from data.

But a single neuron is not yet a neural network: we need a few more steps to get there.

Multiple outputs

Practice problems 01, question 25: generalise our model to an output ***y*** with *l* dimensions. What does the parameter vector become?

$$\theta \in \mathbb{R}^n \quad \longrightarrow \quad \Theta = \left[\theta^{(1)} \;\; \theta^{(2)} \;\; \dots \;\; \theta^{(l)} \right] \in \mathbb{R}^{n \times l}$$

a matrix: each column is the parameter vector of one output

$$y_j = h\left(b_j + \theta^{(j)\top} \boldsymbol{x}\right), \quad j = 1, \dots, l$$

output *j* uses column *j*

$$y_j = h\left(b_j + \sum_i \theta_{ij}\, x_i\right)$$

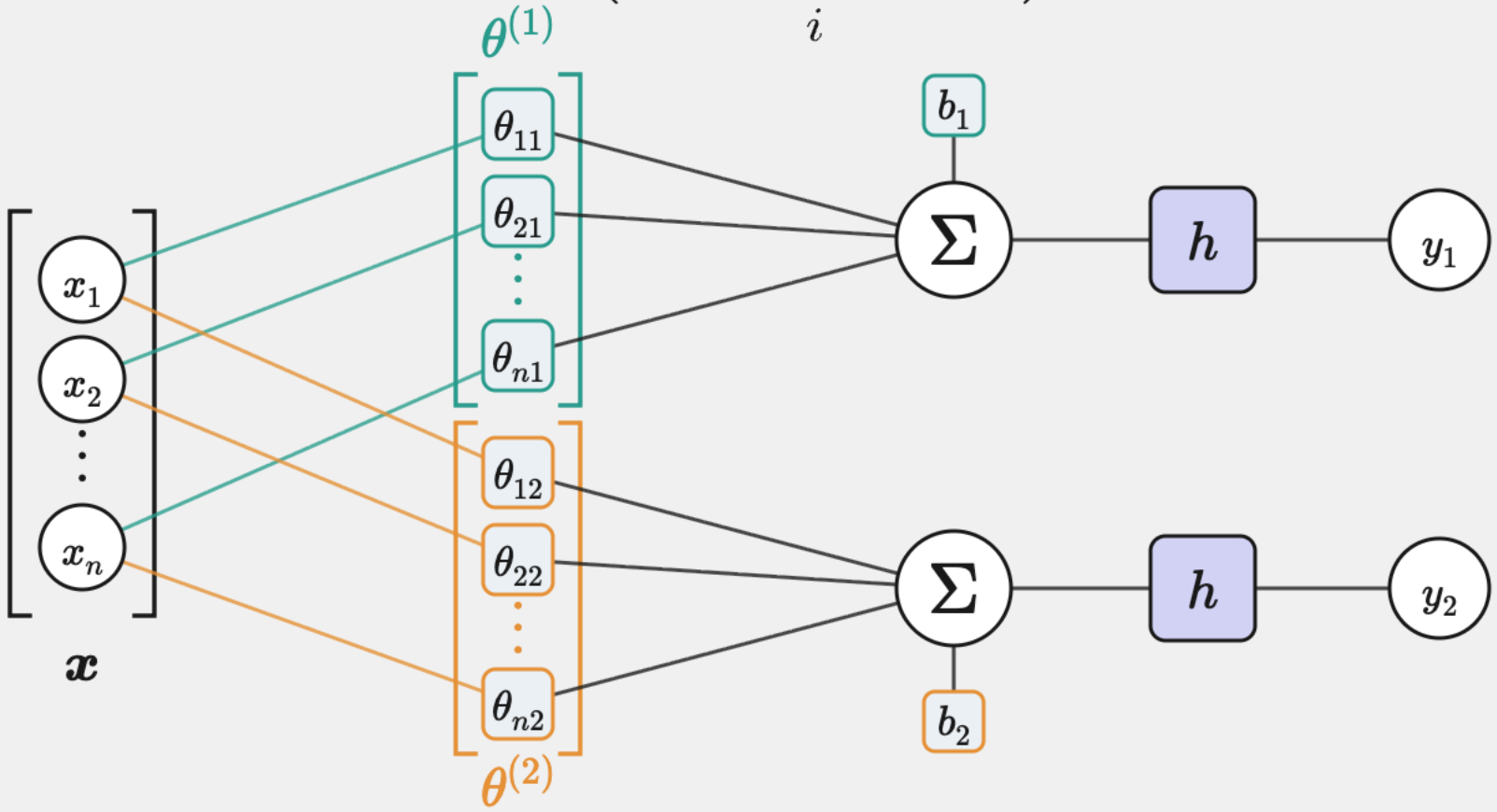
in index notation: θ_{ij} is entry *i* of $\theta^{(j)}$

$$\boldsymbol{W} = \Theta^\top \in \mathbb{R}^{l \times n}, \qquad \boldsymbol{y} = h(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$$

standard: not a list of column vectors but one **weight matrix *W***, one row per output ($W_{ji} = \theta_{ij}$)

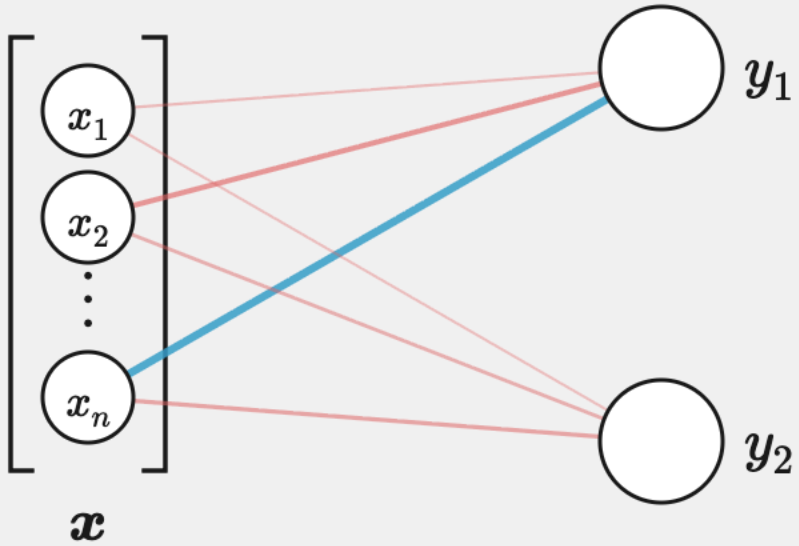
Multiple outputs as a graph

$$y_j = h\left(b_j + \sum_i \theta_{ij}\, x_i\right)$$



Multiple outputs as a graph

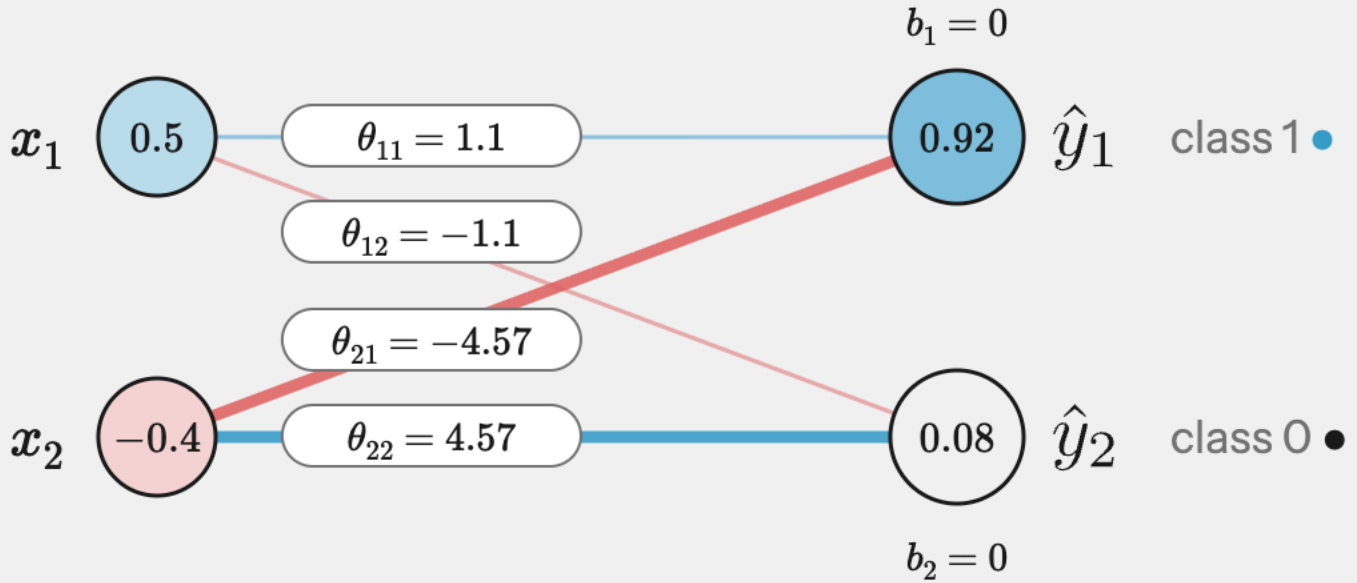
$$y_j = h\left(b_j + \sum_i \theta_{ij}\, x_i\right)$$



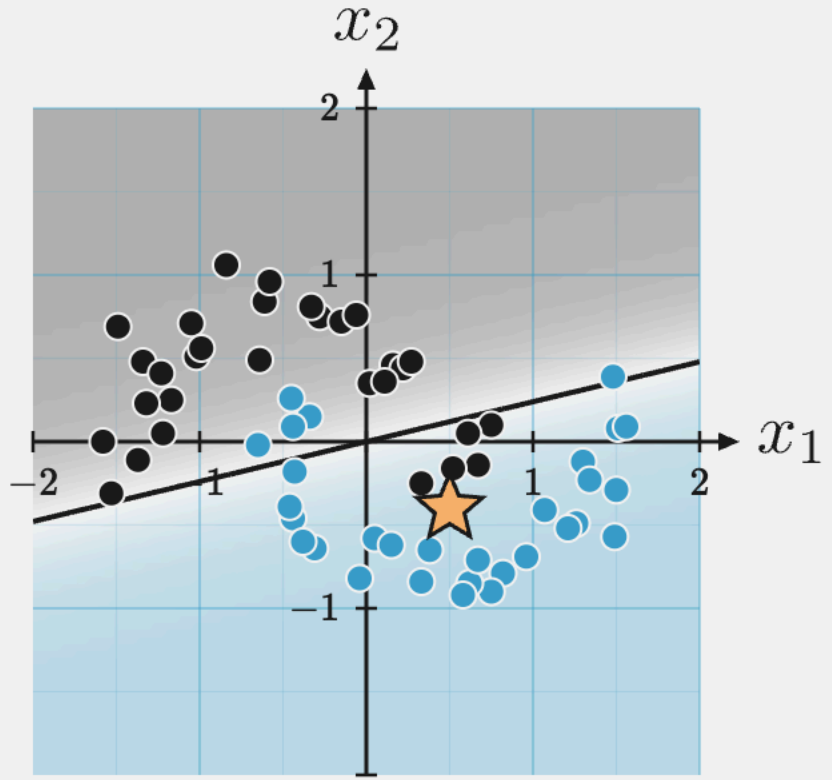
Multiple outputs: still logistic regression

$$\hat{y}_1 = \sigma(b + \theta^\top \boldsymbol{x}), \qquad \hat{y}_2 = \sigma(-b - \theta^\top \boldsymbol{x}) = 1 - \hat{y}_1$$

One output per class: the second neuron's weights are the first's, negated, so this is still logistic regression. Why we prefer a single output: a later lecture.



the two outputs sum to one: $0.92 + 0.08 = 1$

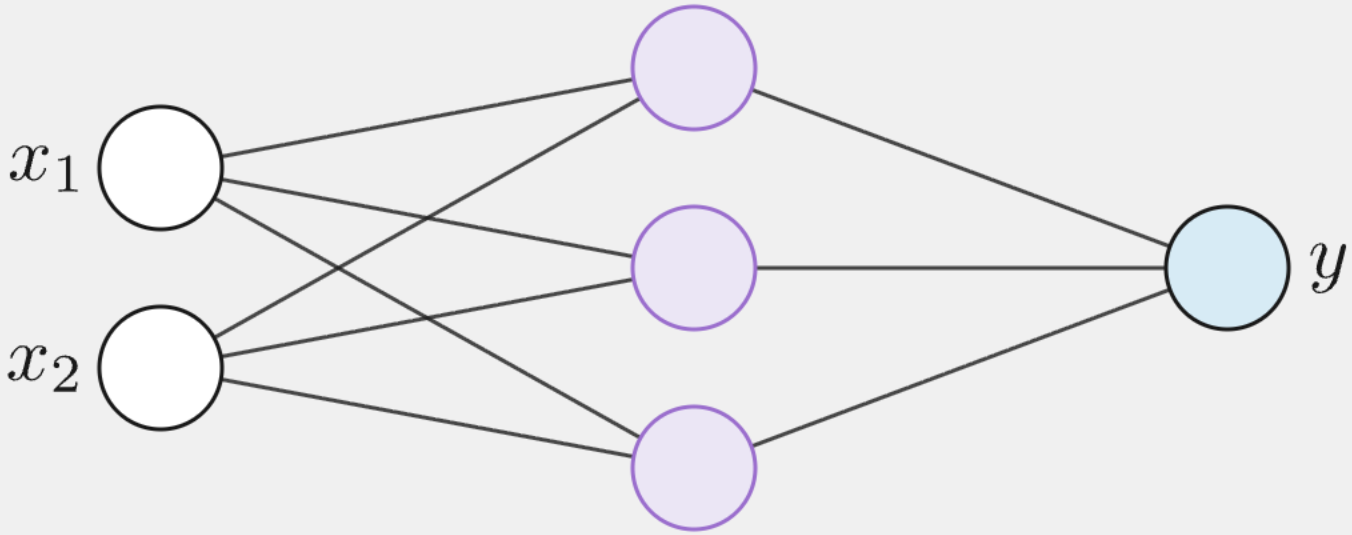


$b = 0.00$



Hidden neurons

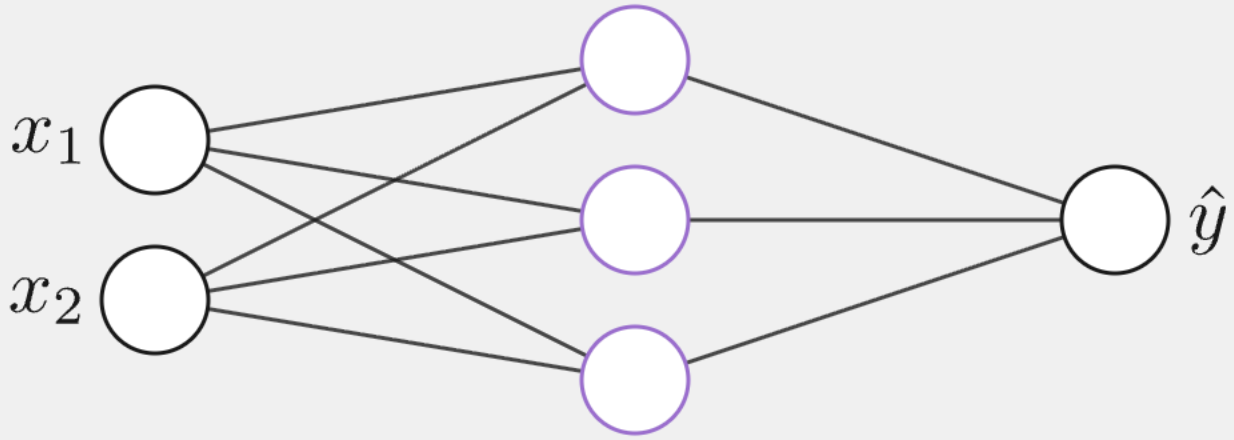
So far our graphs have only input and output nodes, values we observe in our dataset. A **hidden** neuron is one whose value we do not observe; because we never see it, we call it hidden.



inputs: observed hidden: not observed output: observed

The width of a neural network

$$z = h(W^{(1)}x), \qquad \hat{y} = g^{-1}(W^{(2)}z), \qquad z \in \mathbb{R}^k$$



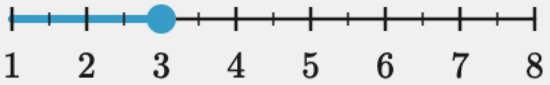
$W^{(1)}$: $k \times 2 = 3 \times 2$

$W^{(2)}$: $1 \times k = 1 \times 3$

9 parameters

input x hidden z: $k = 3$ output

$k = 3$



The depth of a neural network

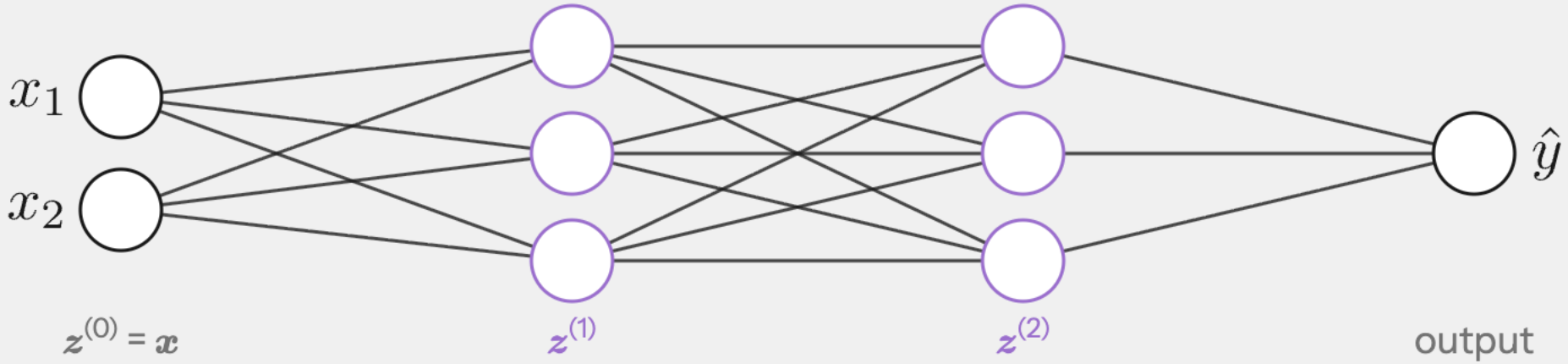
Let $\boldsymbol{z}^{(0)} := \boldsymbol{x}$

$$\zeta^{(i)} = \boldsymbol{W}^{(i)} \boldsymbol{z}^{(i-1)} + \boldsymbol{b}^{(i)}$$

$$\boldsymbol{z}^{(i)} = h(\zeta^{(i)}) \qquad \text{for } i \in \{1, \ldots, L\}$$

$$\hat{\boldsymbol{y}} = g^{-1}(\boldsymbol{W}^{(L+1)} \boldsymbol{z}^{(L)} + \boldsymbol{b}^{(L+1)})$$

an $L = 2$ layer neural network

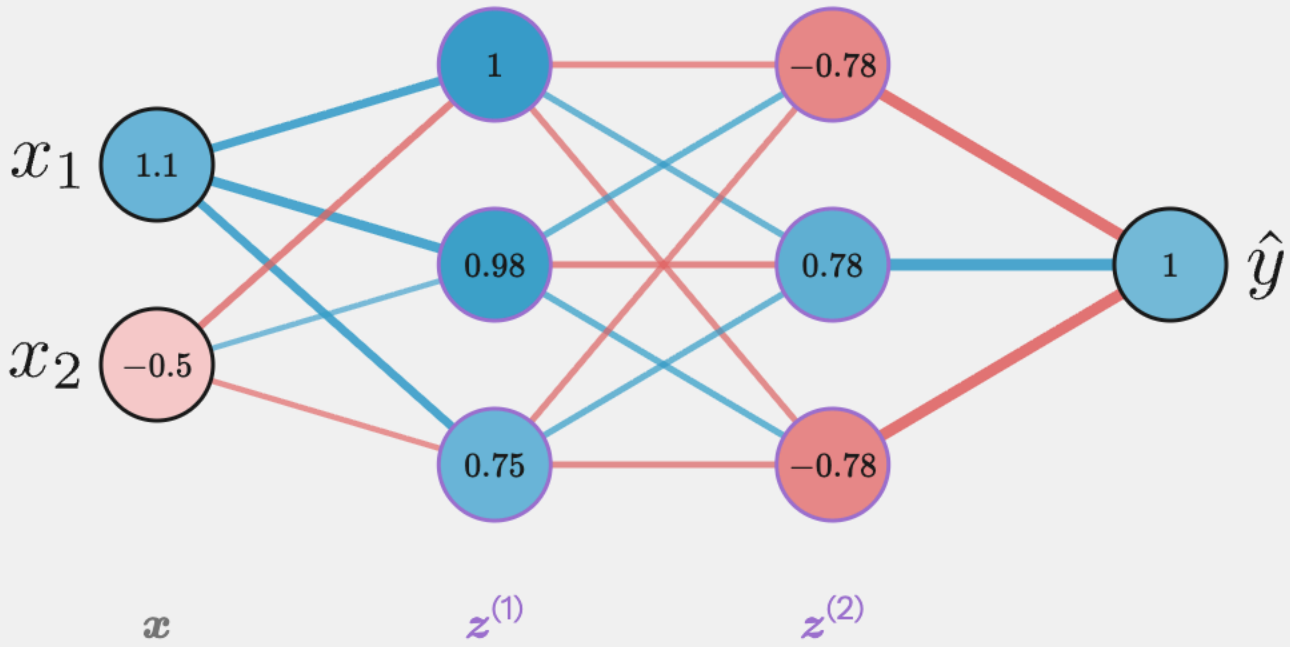


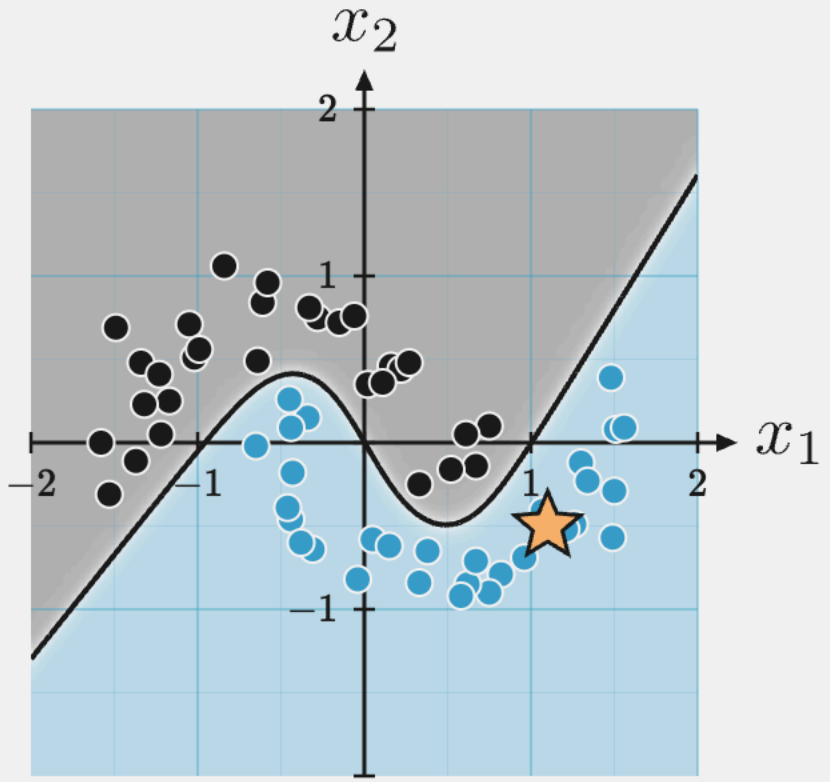
$L = 2$



A neural network on the half moons

$$\boldsymbol{z}^{(i)} = \tanh(\boldsymbol{W}^{(i)}\boldsymbol{z}^{(i-1)} + \boldsymbol{b}^{(i)}), \quad i = 1, 2, \qquad \hat{y} = \sigma(\boldsymbol{W}^{(3)}\boldsymbol{z}^{(2)} + \boldsymbol{b}^{(3)})$$

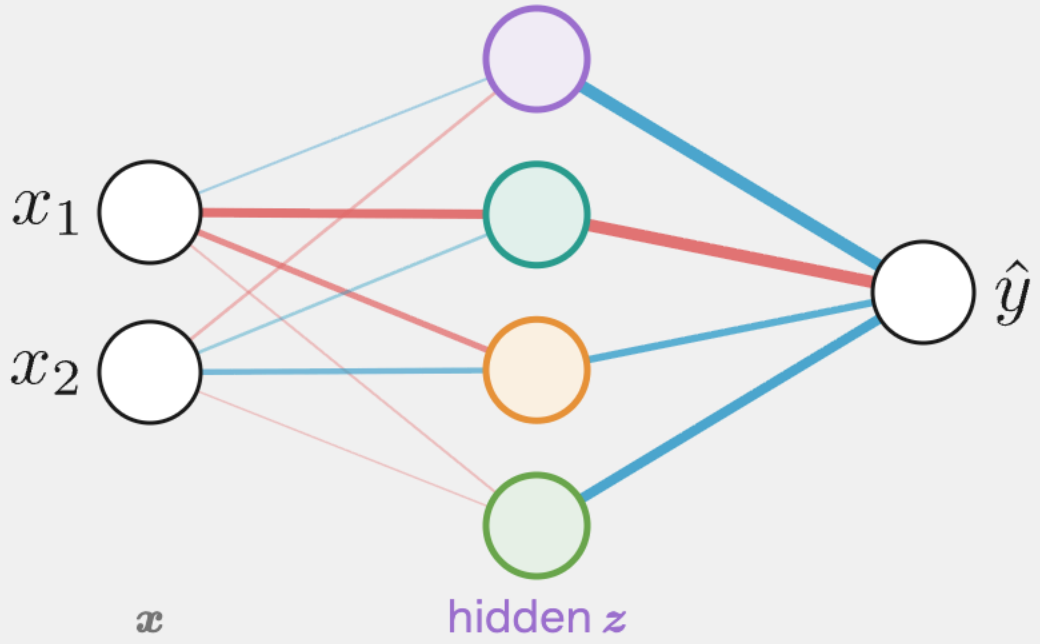


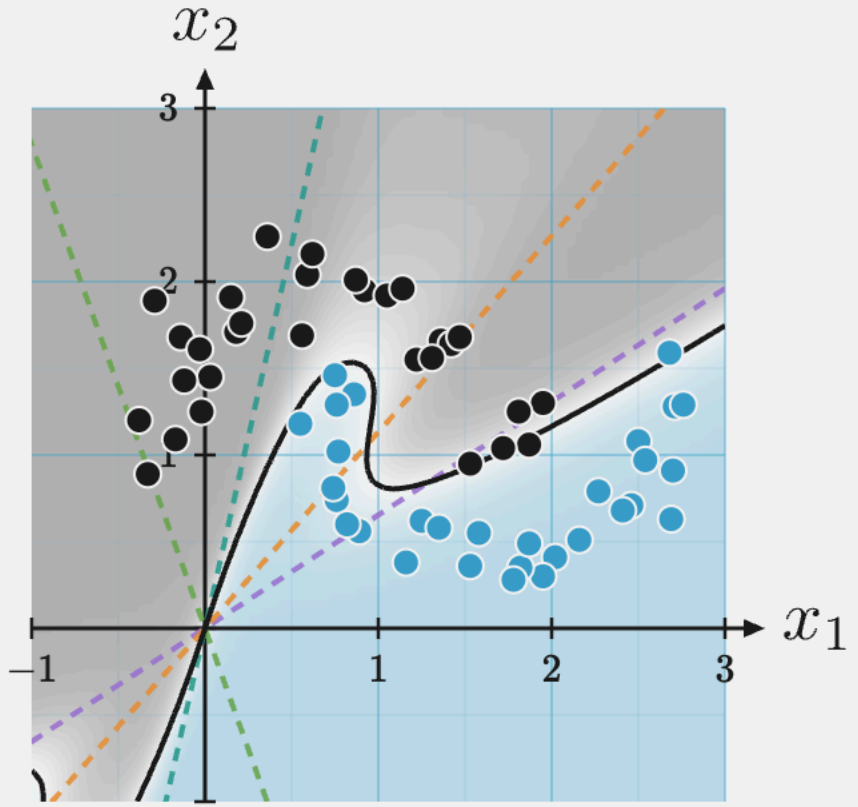


output 1: class 1 training accuracy 100%

MLP and basis expansion

$$\hat{y} = \sigma(\boldsymbol{W}^{(2)}\boldsymbol{z}), \qquad \boldsymbol{z} = \tanh\left(\boldsymbol{W}^{(1)}\phi(\boldsymbol{x})\right), \qquad \phi(\boldsymbol{x}) = \boldsymbol{x} \ \text{(no basis expansion)}$$





$k = 4$: training accuracy 97%

no biases: the boundary still passes through the origin, but now it can bend

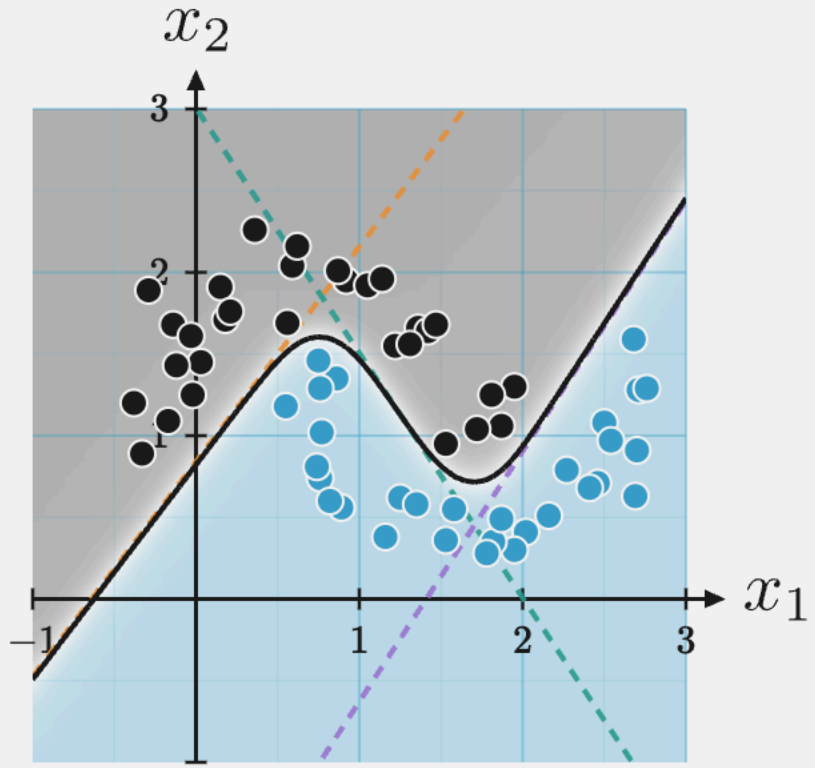
$k = 4$



Learned bias versus inductive bias

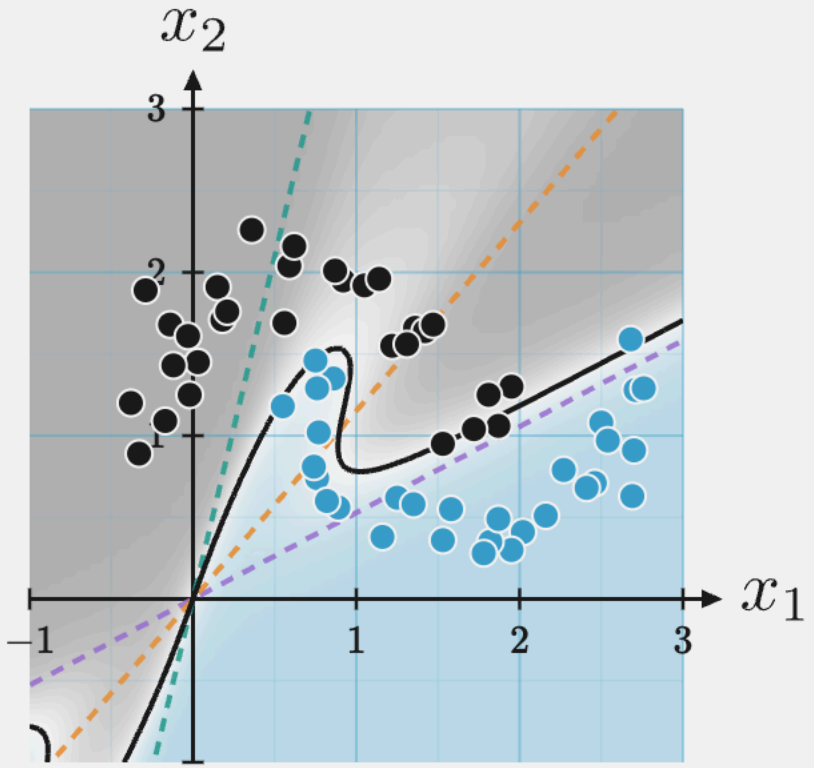
$$\hat{y} = \sigma(\boldsymbol{W}^{(2)}\boldsymbol{z}), \quad \boldsymbol{z} = \tanh(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$

with a learned bias

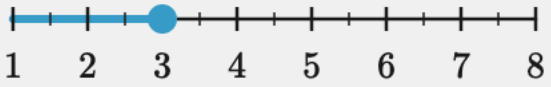


$$\hat{y} = \sigma(\boldsymbol{W}^{(2)}\boldsymbol{z}), \quad \boldsymbol{z} = \tanh(\boldsymbol{W}^{(1)}\boldsymbol{x})$$

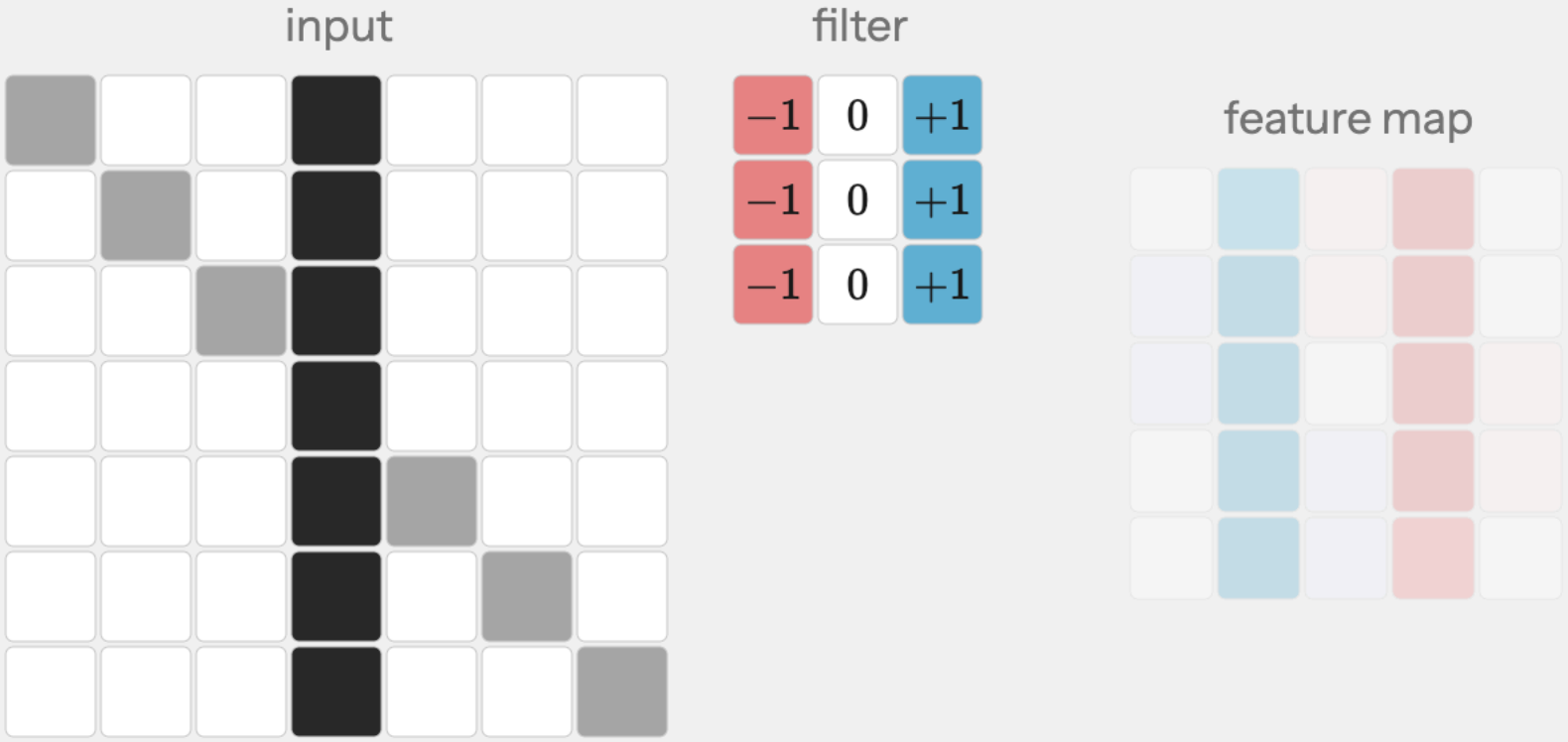
without a bias

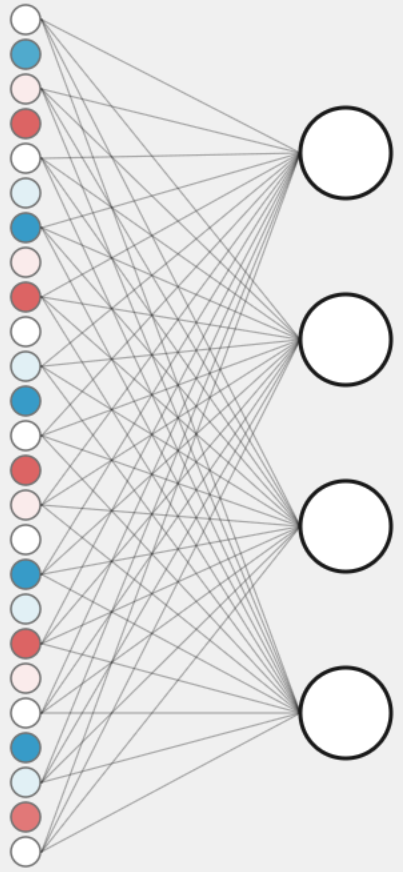


$$k = 3$$



More complex inductive biases





Learning in an MLP

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)})^\top$$

Gradient Descent Algorithm:

1. Randomly initialize our model
2. Take gradient of loss function w/data
3. Update model parameters w/grad
4. Repeat 2 and 3 ‘k’ times

Gradient computations

$$
\begin{aligned}
\boldsymbol{z}^{(0)} &= \boldsymbol{x} \\
\boldsymbol{\zeta}^{(1)} &= \boldsymbol{W}^{(1)}\boldsymbol{z}^{(0)} + \boldsymbol{b}^{(1)} \\
\boldsymbol{z}^{(1)} &= \tanh(\boldsymbol{\zeta}^{(1)}) \\
\boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + \boldsymbol{b}^{(2)} \\
\mathcal{L}(\boldsymbol{y}, \boldsymbol{z}^{(2)}) &= (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2
\end{aligned}
$$

Gradient computations

$$
\begin{aligned}
\boldsymbol{z}^{(0)} &= \boldsymbol{x} \\
\boldsymbol{\zeta}^{(1)} &= \boldsymbol{W}^{(1)}\boldsymbol{z}^{(0)} + \boldsymbol{b}^{(1)} \\
\boldsymbol{z}^{(1)} &= \tanh(\boldsymbol{\zeta}^{(1)}) \\
\boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + \boldsymbol{b}^{(2)}
\end{aligned}
$$

$$
\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(2)}} \frac{\partial \boldsymbol{z}^{(2)}}{\partial \boldsymbol{z}^{(1)}} \frac{\partial \boldsymbol{z}^{(1)}}{\partial \boldsymbol{\zeta}^{(1)}} \frac{\partial \boldsymbol{\zeta}^{(1)}}{\partial \boldsymbol{W}^{(1)}}
$$

$$
= 2\Big(\boldsymbol{W}^{(2)\top}(\boldsymbol{z}^{(2)} - \boldsymbol{y}) \circ \big(1 - \tanh^2(\boldsymbol{\zeta}^{(1)})\big)\Big)\, \boldsymbol{z}^{(0)\top}
$$

Considering the form of the gradient

$$
\begin{aligned}
\boldsymbol{z}^{(0)} &= \boldsymbol{x} \\
\zeta^{(1)} &= \boldsymbol{W}^{(1)}\boldsymbol{z}^{(0)} + \boldsymbol{b}^{(1)} \\
\boldsymbol{z}^{(1)} &= \tanh(\zeta^{(1)}) \\
\boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + \boldsymbol{b}^{(2)} \\
\mathcal{L}(\boldsymbol{y}, \boldsymbol{z}^{(2)}) &= (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2
\end{aligned}
$$

$$
\mathcal{L}(\theta) \longrightarrow f : \mathbb{R}^n \to \mathbb{R}
$$

For a function of a vector we have

$$f : \mathbb{R}^n \to \mathbb{R}$$

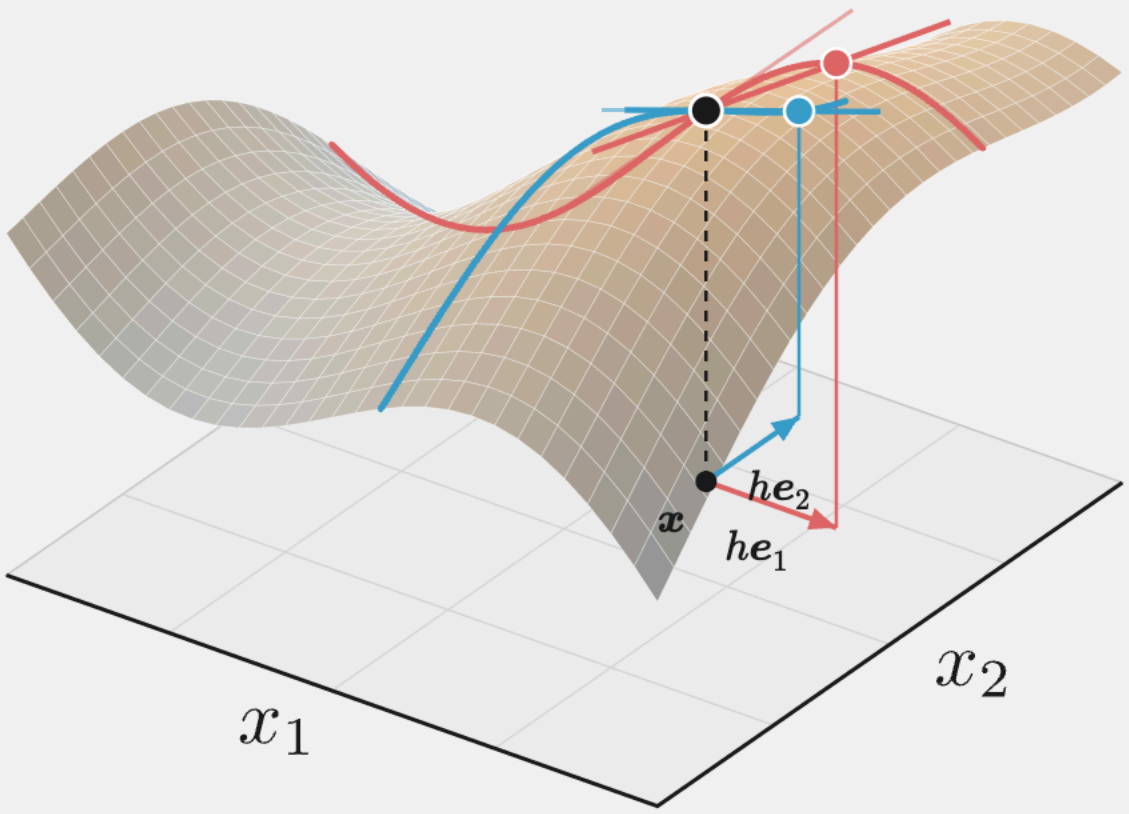
$$\frac{\partial f}{\partial x_1} = \lim_{h \to 0} \frac{f(x_1 + h, x_2, \ldots, x_n) - f(\boldsymbol{x})}{h},$$

$$\frac{\partial f}{\partial x_2} = \lim_{h \to 0} \frac{f(x_1, x_2 + h, \ldots, x_n) - f(\boldsymbol{x})}{h},$$

$$\vdots$$

$$\frac{\partial f}{\partial x_n} = \lim_{h \to 0} \frac{f(x_1, x_2, \ldots, x_n + h) - f(\boldsymbol{x})}{h}$$

Numerical differentiation – Finite differences



$$\frac{f(\boldsymbol{x}+h\boldsymbol{e}_1)-f(\boldsymbol{x})}{h} \approx \frac{\partial f}{\partial x_1}$$

$= 0.54 \quad \partial f/\partial x_1 = 0.78$

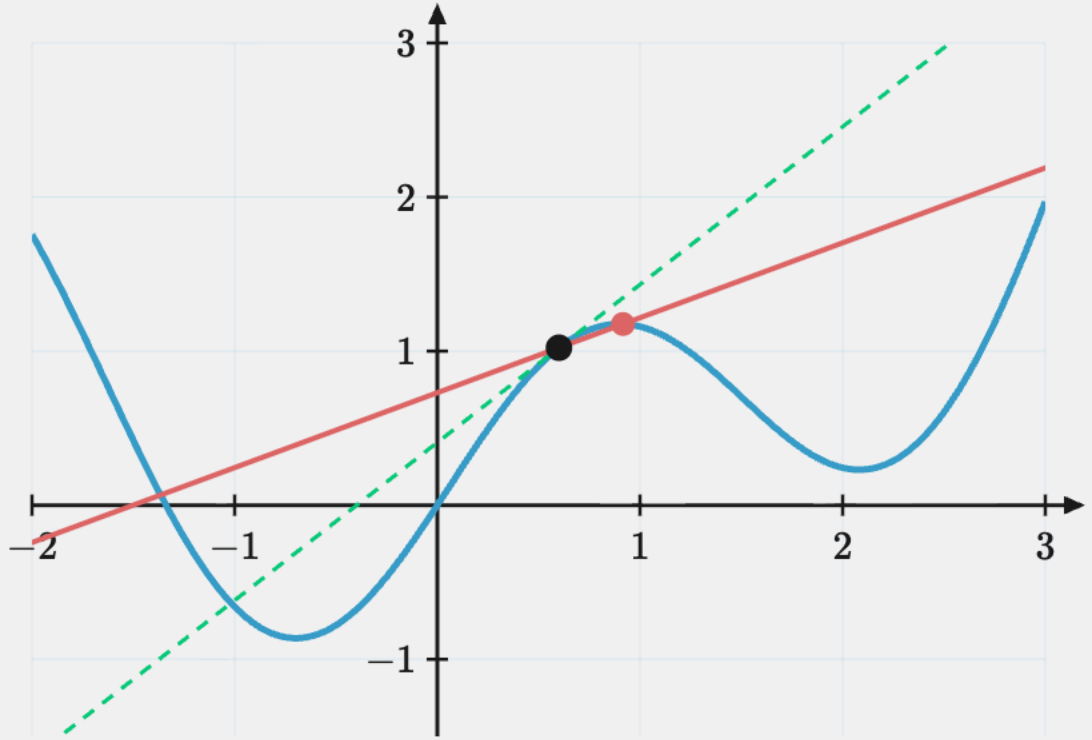
$$\frac{f(\boldsymbol{x}+h\boldsymbol{e}_2)-f(\boldsymbol{x})}{h} \approx \frac{\partial f}{\partial x_2}$$

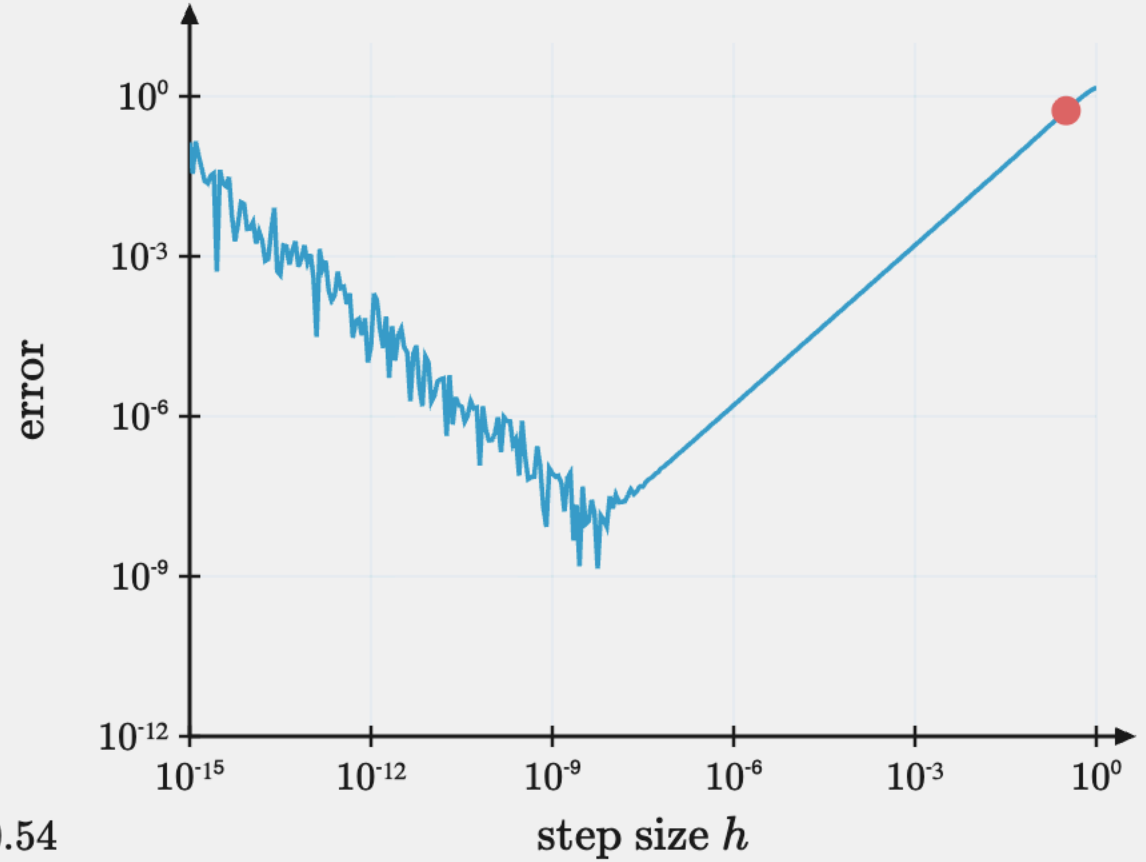
$= -0.38 \quad \partial f/\partial x_2 = -0.38$

$h = 0.80$

0.5 1 1.5

Numerical differentiation – Finite differences





finite difference 0.486330927 true $f\prime(x)$ 1.024715509 error 0.54

$log_{10}\, h = -0.50$



For a function of a vector we have

$$\mathcal{L}(\theta) : \mathbb{R}^n \to \mathbb{R}$$

Though we are nominally interested in this loss function, we will consider an arbitrary function f (wlog) in the rest of the slides

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \lim_{h \to 0} \frac{\mathcal{L}(\theta_1 + h, \theta_2, \ldots, \theta_n) - \mathcal{L}(\theta)}{h},$$

$$\frac{\partial \mathcal{L}}{\partial \theta_2} = \lim_{h \to 0} \frac{\mathcal{L}(\theta_1, \theta_2 + h, \ldots, \theta_n) - \mathcal{L}(\theta)}{h},$$

$$\vdots$$

$$\frac{\partial \mathcal{L}}{\partial \theta_n} = \lim_{h \to 0} \frac{\mathcal{L}(\theta_1, \theta_2, \ldots, \theta_n + h) - \mathcal{L}(\theta)}{h}.$$

Forward mode automatic differentiation

Approximating (and incurring error) for how the function changes to such small changes is not really necessary when we know the derivatives of each of the intermediate operations.

We have the chain rule!

$$f(\boldsymbol{x}) = h(g(\boldsymbol{x}))$$

$$\boldsymbol{J}_f = \boldsymbol{J}_{h \circ g} = \boldsymbol{J}_h(g(\boldsymbol{x}))\, \boldsymbol{J}_g(\boldsymbol{x})$$

Forward mode automatic differentiation

$$
\begin{aligned}
\boldsymbol{z}^{(0)} &:= \boldsymbol{x} \\
\zeta^{(i)} &= \boldsymbol{W}^{(i)} \boldsymbol{z}^{(i-1)} + \boldsymbol{b}^{(i)} \\
\boldsymbol{z}^{(i)} &= h(\zeta^{(i)}) \qquad \text{for } i \in \{1, \ldots, L\} \\
\hat{\boldsymbol{y}} &= g^{-1}(\boldsymbol{W}^{(L+1)} \boldsymbol{z}^{(L)} + \boldsymbol{b}^{(L+1)})
\end{aligned}
$$

in general: an L layer network (L hidden layers)

$$
\boldsymbol{J} = \boldsymbol{J}^{(L)} \boldsymbol{J}^{(L-1)} \ldots \boldsymbol{J}^{(2)} \boldsymbol{J}^{(1)}
$$

its derivative: a chain of Jacobians

Forward mode automatic differentiation

For a function *f* of *n* inputs and *m* outputs, what is the Jacobian times a basis vector?

$$\boldsymbol{J} \in \mathbb{R}^{m \times n}, \quad J_{kj} = \frac{\partial f_k}{\partial x_j}, \qquad \boldsymbol{e}_i = [0, \ldots, 0, 1, 0, \ldots, 0]^\top$$

$$(\boldsymbol{J}\boldsymbol{e}_i)_k = \sum_j J_{kj} \, (\boldsymbol{e}_i)_j$$

$$= J_{ki}$$

$(\boldsymbol{e}_i)_j$ is 1 when $j = i$ and 0 otherwise, so the sum isolates the term $j = i$

$$\boldsymbol{J}\boldsymbol{e}_i = [J_{1i}, \ldots, J_{mi}]^\top = \text{column } i \text{ of } \boldsymbol{J}$$

multiplying by $\boldsymbol{e}_i$ picks out the i-th column

$$= \frac{\partial f}{\partial x_i} \approx \frac{f(\boldsymbol{x} + h\boldsymbol{e}_i) - f(\boldsymbol{x})}{h}$$

the finite difference approximates exactly this column

Forward mode automatic differentiation

$$\boldsymbol{J}\boldsymbol{e}_i = \boldsymbol{J}^{(L)}\,\boldsymbol{J}^{(L-1)}\,\ldots\,\boldsymbol{J}^{(2)}\,\boldsymbol{J}^{(1)}\,\boldsymbol{e}_i$$

Forward mode AD stems from the observation that we do not need to compute the full ***J*** to get the *i*th partial derivative: we can just compute each of these intermediate matrix-vector products.

$$\begin{aligned}
\boldsymbol{J}\boldsymbol{e}_i &= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)}\ldots\boldsymbol{J}^{(2)}\left(\boldsymbol{J}^{(1)}\boldsymbol{e}_i\right) \\
&= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)}\ldots\boldsymbol{J}^{(3)}\left(\boldsymbol{J}^{(2)}\boldsymbol{v}^{(1)}\right) \\
&= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)}\ldots\boldsymbol{J}^{(4)}\left(\boldsymbol{J}^{(3)}\boldsymbol{v}^{(2)}\right) \\
&\vdots \\
&= \boldsymbol{J}^{(L)}\boldsymbol{v}^{(L-1)}
\end{aligned}$$

Example of forward mode

```
class AdVar:
    '''
    We want to keep track of both the value and
    the derivative of our variable, so it will
    hold two values one representing each
    '''
    def __init__(self, **kwargs):
        self.val = 0
        self.der = 1

        # re-assign these default values
        if 'val' in kwargs:
            self.val = kwargs['val']
        if 'der' in kwargs:
            self.der = kwargs['der']
```

Example of forward mode

```
class AdVar:
    '''
    We want to keep track of both the value and
    the derivative of our variable, so it will
    hold two values one representing each
    '''
    def __init__(self, **kwargs):
        self.val = 0
        self.der = 1

        # re-assign these default values
        if 'val' in kwargs:
            self.val = kwargs['val']
        if 'der' in kwargs:
            self.der = kwargs['der']
```

```
def add(a, b):
    # Create output evaluation and derivative object
    c = AdVar()
    # switch to determine if a or b is a constant
    if type(a) != AdVar:
        c.val = a + b.val
        c.der = b.der
    elif type(b) != AdVar:
        c.val = a.val + b
        c.der = a.der
    else:
        c.val = a.val + b.val
        c.der = a.der + b.der
    return c
```

Instance of forward mode

Consider our NN as a sequence of functions we could program:

$$\boldsymbol{z}^{(2)} = \text{Matmul}\Big(\boldsymbol{W}^{(2)}, \tanh\left(\text{Matmul}(\boldsymbol{W}^{(1)}, \boldsymbol{z}^{(0)})\right)\Big)$$

$$\mathcal{L} = (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2$$

Operation	**Value Update**	**Derivative Update**
Addition of a constant c	$g(w) + c$	$\frac{d}{dw}(g(w) + c) = \frac{d}{dw} g(w)$
Multiplication by a constant c	$c\, g(w)$	$\frac{d}{dw}(c\, g(w)) = c \frac{d}{dw} g(w)$
Raising to a power n	$g(w)^n$	$\frac{d}{dw}(g(w)^n) = n\, g(w)^{n-1} \frac{d}{dw} g(w)$
Applying tanh	$\tanh(g(w))$	$\frac{d}{dw} \tanh(g(w)) = \left(1 - \tanh^2(g(w))\right) \frac{d}{dw} g(w)$

Instance of forward mode

$$\boldsymbol{z}^{(2)} = \mathrm{Matmul}\Big(\boldsymbol{W}^{(2)}, \tanh\left(\mathrm{Matmul}(\boldsymbol{W}^{(1)}, \boldsymbol{z}^{(0)})\right)\Big)$$

$$\mathcal{L} = (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2$$

Value	Derivative
$x_0 = x$	$d_0 = 1$
$x_1 = W^{(1)} x_0$	$d_1 = W^{(1)} d_0$
$x_2 = \tanh(x_1)$	$d_2 = \left(1 - \tanh^2(x_1)\right) d_1$
$x_3 = W^{(2)} x_2$	$d_3 = W^{(2)} d_2$
$x_4 = x_3 - y$	$d_4 = d_3$
$x_5 = (x_4)^2$	$d_5 = 2x_4\, d_4$

Running forward mode

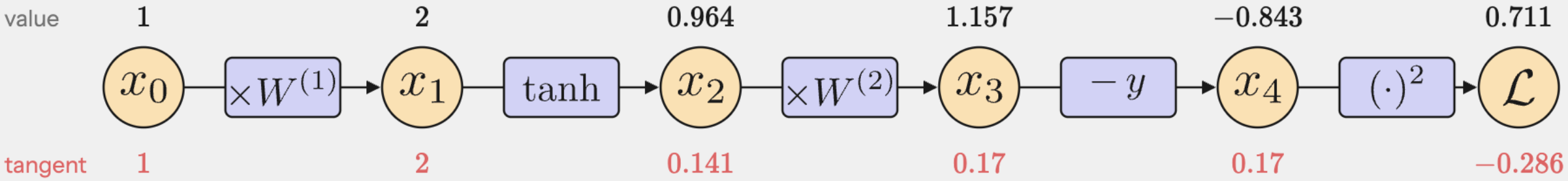
$$\boldsymbol{z}^{(2)} = \text{Matmul}\Big(\boldsymbol{W}^{(2)}, \tanh\left(\text{Matmul}(\boldsymbol{W}^{(1)}, \boldsymbol{z}^{(0)})\right)\Big)$$

$$\mathcal{L} = (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2$$

$$x = 1,\ y = 2,\ W^{(1)} = 2,\ W^{(2)} = 1.2$$

$x_0 = 1$	$d_0 = 1$
$x_1 = 2 \cdot 1 = 2$	$d_1 = 2 \cdot d_0 = 2$
$x_2 = \tanh(2) \approx 0.964$	$d_2 = (1 - \tanh^2(2))\, d_1 \approx 0.141$
$x_3 = 1.2\, x_2 \approx 1.157$	$d_3 = 1.2\, d_2 \approx 0.170$
$x_4 = x_3 - 2 \approx -0.843$	$d_4 = d_3 \approx 0.170$
$x_5 = x_4^2 \approx 0.711$	$d_5 = 2x_4\, d_4 \approx -0.286$

Forward mode, live



one forward pass: $\mathrm{d}\mathcal{L}/\mathrm{d}x = -0.286$ (for $\partial\mathcal{L}/\partial W^{(1)}$: another pass, seeded on $W^{(1)}$)

$x = 1.00$



$W^{(1)} = 2.00$



$y = 2.00$



$W^{(2)} = 1.20$



Complexity of forward mode

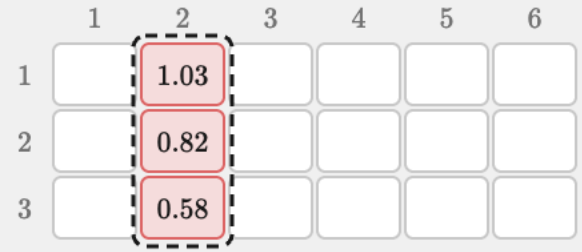
Letting the forward pass complexity be ‘T’, the computational complexity of computing the column of the Jacobian with forward mode automatic differentiation is:

$$O(2nT)$$

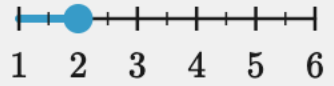
$J^{(3)}$ $J^{(2)}$ $J^{(1)}$

3×4 4×5 5×6

J (3×6)



$i = 2$



one pass: $30 + 20 + 12 = 62$ multiply-adds for column 2; J so far: 1 of 6 columns (1 pass)

Reverse or Adjoint Mode

The idea of reverse mode is to instead of beginning with an input, begin with an output and propagate backwards through the composition of functions:

$$\bar{\boldsymbol{y}}\boldsymbol{J} = (\bar{\boldsymbol{y}}\boldsymbol{J}^{(L)})\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(2)}\boldsymbol{J}^{(1)}$$

Reverse or Adjoint Mode

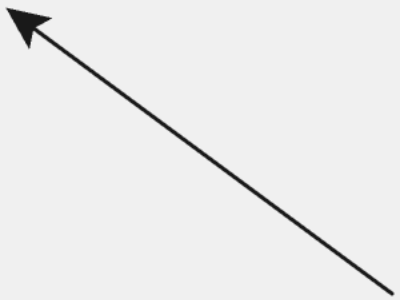
The idea of reverse mode is to instead of beginning with an input, begin with an output and propagate backwards through the composition of functions:

$$
\begin{aligned}
\bar{\boldsymbol{y}}\boldsymbol{J} &= \left(\bar{\boldsymbol{y}}\boldsymbol{J}^{(L)}\right)\boldsymbol{J}^{(L-1)}\dots\boldsymbol{J}^{(1)} \\
&= \left(\bar{\boldsymbol{y}}^{(L)}\boldsymbol{J}^{(L-1)}\right)\boldsymbol{J}^{(L-2)}\dots\boldsymbol{J}^{(1)} \\
&\dots \\
&= \bar{\boldsymbol{y}}^{(2)}\boldsymbol{J}^{(1)}
\end{aligned}
$$

Example of reverse mode

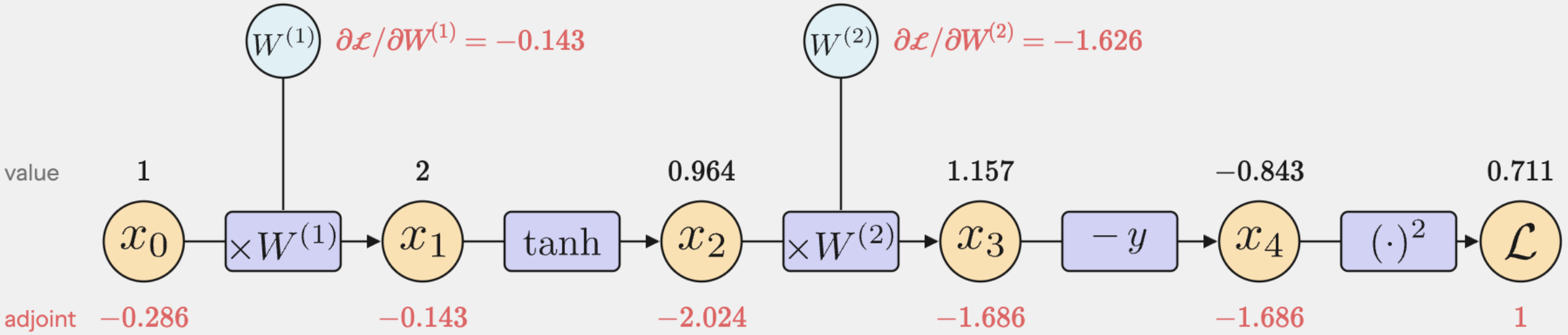
When we propagate backwards through computations, we must be careful to ensure gradients are accumulated appropriately. Walking backwards is a bit more difficult than walking forward.

$$\bar{\boldsymbol{y}}\boldsymbol{J} = (\bar{\boldsymbol{y}}\boldsymbol{J}^{(L)})\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(2)}\boldsymbol{J}^{(1)}$$



For example, here we have already assumed we have an output. This is the result of a forward computation

Reverse mode on our network



one backward pass: $\partial\mathcal{L}/\partial W^{(1)} = -0.143,\ \partial\mathcal{L}/\partial W^{(2)} = -1.626,\ \partial\mathcal{L}/\partial x = -0.286$

$x = 1.00$



$W^{(1)} = 2.00$



$y = 2.00$



$W^{(2)} = 1.20$



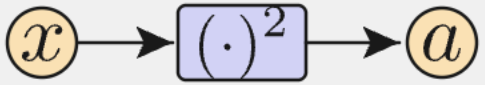
Computational graphs

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos\left(x^2 + \exp(x^2)\right)$$

1. Identify the elementary operations and look for the ones that occur most and for shared structure
2. We will have variable nodes leading to operations.
3. Start with the most common operations and then build the graph outwards

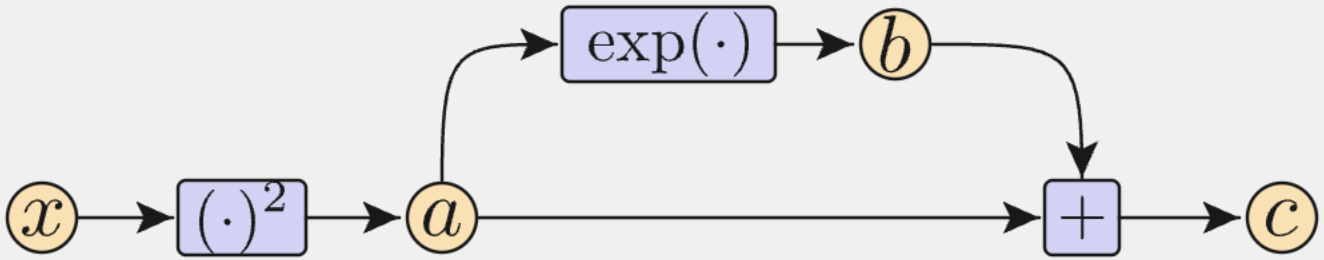
Computational graphs

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos\left(x^2 + \exp(x^2)\right)$$

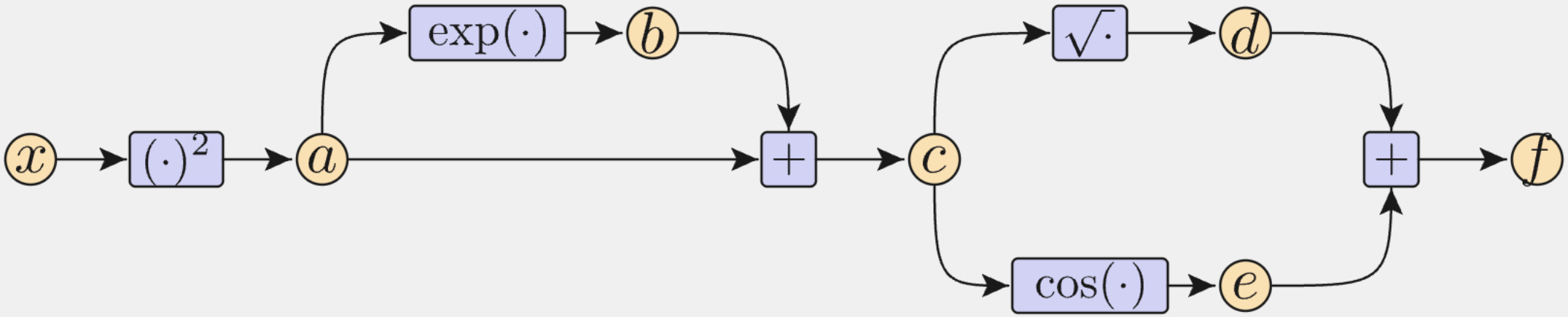


Computational graphs

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos\left(x^2 + \exp(x^2)\right)$$

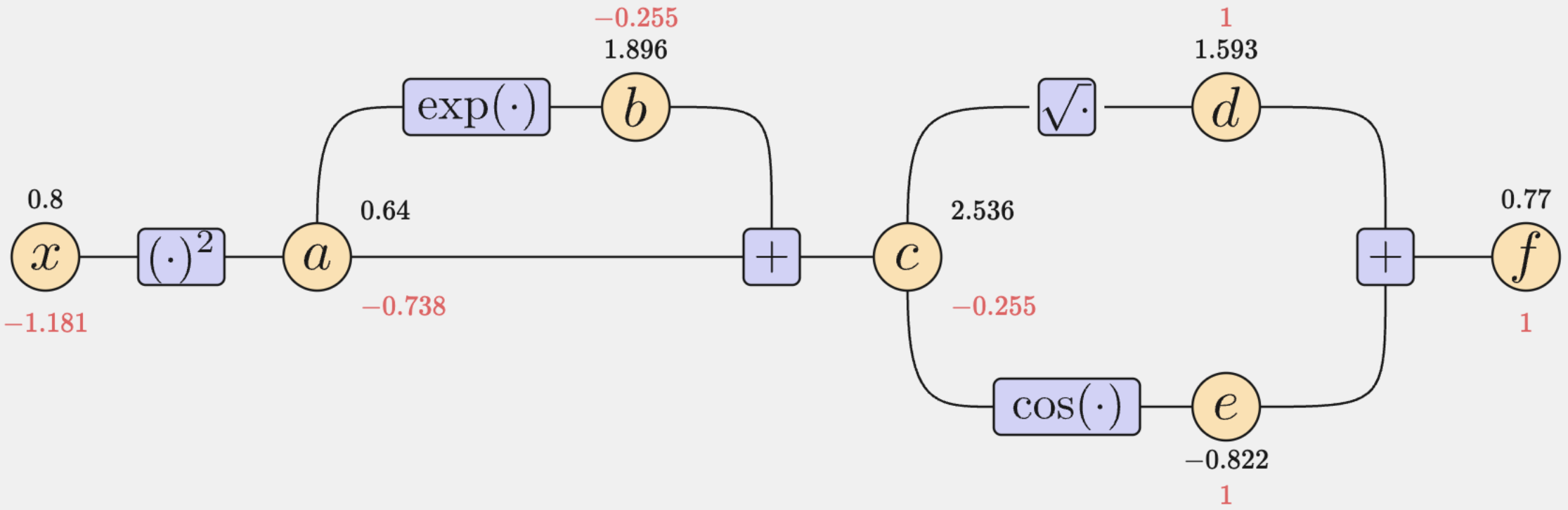


Computational graphs



$$\frac{\partial f}{\partial x_i} = \sum_{x_j : x_i \in \mathrm{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i} = \sum_{x_j : x_i \in \mathrm{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial g_j}{\partial x_i}$$

Example of reverse mode

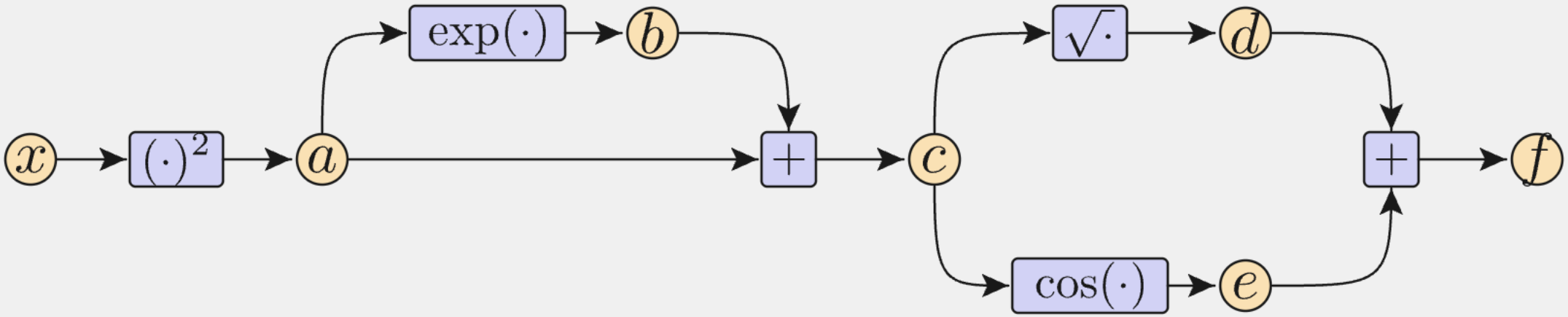


reverse mode: $\partial \mathrm{f}/\partial \mathrm{x} = -1.18134$ (finite difference: -1.18134)

$x = 0.80$



Computational graphs



$$\frac{\partial a}{\partial x} = 2x$$
$$\frac{\partial b}{\partial a} = \exp(a)$$
$$\frac{\partial c}{\partial a} = 1 = \frac{\partial c}{\partial b}$$

$$\frac{\partial d}{\partial c} = \frac{1}{2\sqrt{c}}$$
$$\frac{\partial e}{\partial c} = -\sin(c)$$
$$\frac{\partial f}{\partial d} = 1 = \frac{\partial f}{\partial e}$$

Complexity of Reverse Mode

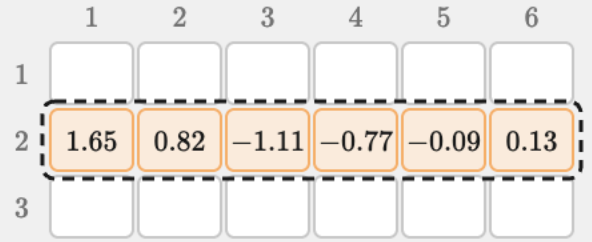
One must first evaluate the function, incurring a cost of T. Then for each output one computes a backwards pass which allows us to compute the Jacobian in:

$$O(mT + T)$$

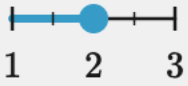
$J^{(3)}$ $J^{(2)}$ $J^{(1)}$

3×4 4×5 5×6

J (3×6)



$k = 2$



one pass: $12 + 20 + 30 = 62$ multiply-adds for row 2; J so far: 1 of 3 rows (1 pass)

Auto. diff. complexity

Forward mode complexity: one column of the Jacobian matrix at a time.

$$O(2nT)$$

Reverse mode complexity: one row of the Jacobian matrix at a time.

$$O(mT + T)$$

Which one is preferable when computing a loss?

Feedback



https://forms.gle/JS1qqjU6CEwiGA5v6

Please let me know your thoughts and concerns!